

System 13: High-Throughput Logging Framework

Zero-Prerequisite Intuition: The "Airport Black Box" Metaphor What is a logging framework, and why can't we just use `print()`? Imagine an airplane in mid-flight. Hundreds of sensors (engine temperature, altitude, hydraulic pressure) record telemetry every millisecond. If the pilot had to stop flying the airplane and personally hand-write every temperature reading on paper before pulling the steering wheel, the plane would crash!

Instead, sensors dump readings into an automated **Black Box** in the background while the pilot continues flying uninterrupted.

In high-scale web servers, every request generates logs. If an application thread blocks on a synchronous disk write (`file.write()`) on every request, your server throughput drops from 10,000 requests/sec to 100 requests/sec because hard drives and networks are 100,000x slower than CPU RAM!

The **#1 Low-Level Design question at Uber, Stripe, and Google** is: *"Design a multi-threaded, asynchronous, high-throughput logging library that supports log levels, custom formatters, multiple destinations (Console, File, S3), and never blocks worker threads."*

1. Requirements & System Boundaries

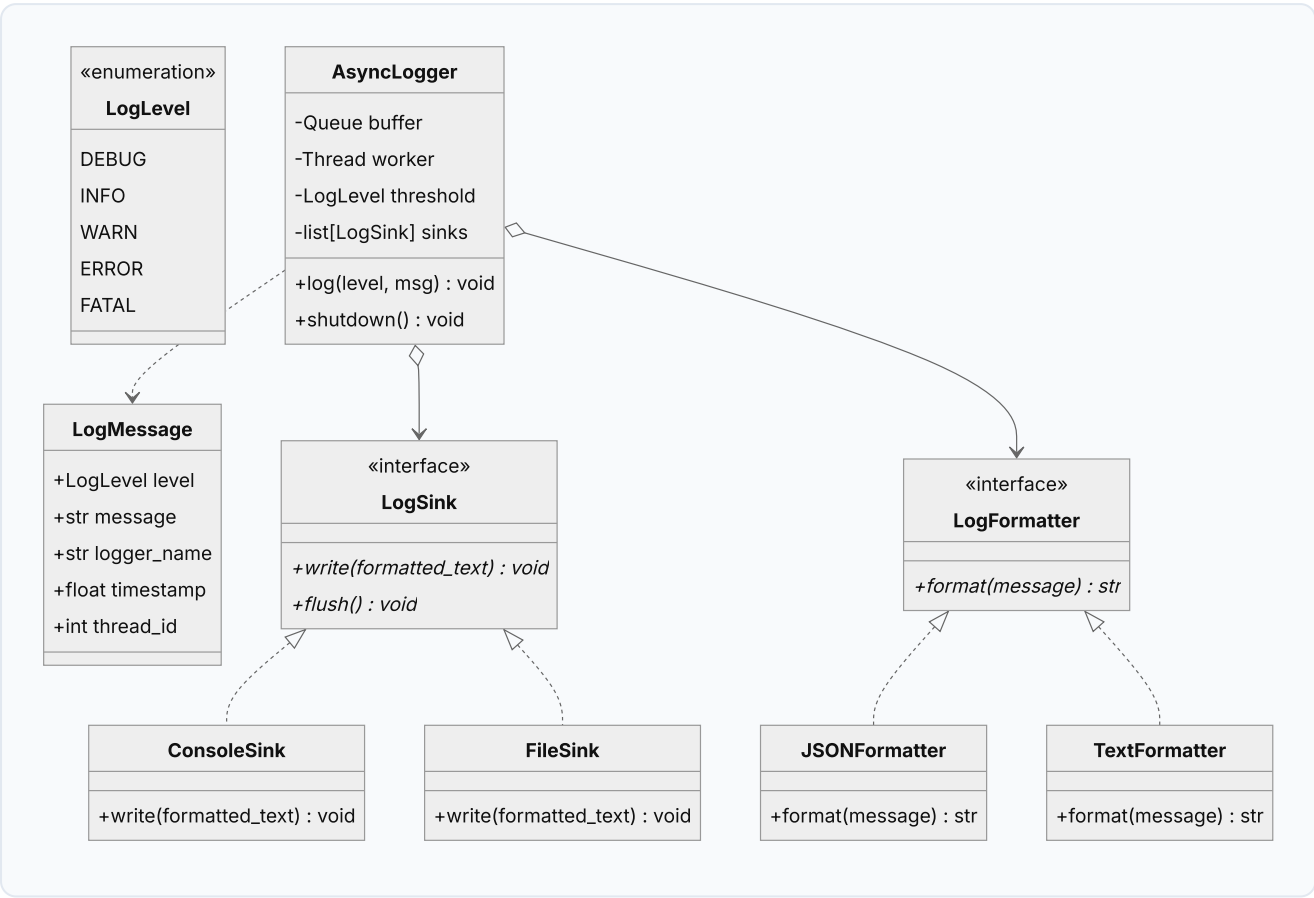
Functional Requirements

1. **Log Levels:** Strict priority filtering: `DEBUG (10) < INFO (20) < WARN (30) < ERROR (40) < FATAL (50)`.
2. **Pluggable Appenders / Sinks:** Support writing to multiple destinations simultaneously (Console, File, Remote Network).
3. **Pluggable Formatters:** Support converting log records into Human-Readable Plain Text or Machine-Readable JSON.

Non-Functional Requirements

- **Zero Latency Overhead (Asynchronous):** Worker threads log in $< 1\mu s$ by pushing into an in-memory queue. A background thread handles batch flushing to disk.
- **Buffer Overflow Policy:** What happens when logs arrive faster than disk can write? (Drop lowest priority logs vs. block producer).

2. Design Patterns Architecture



3. Complete Python Implementation

4. Verification & Stress-Testing